

Session 16

GitHub Collaboration, PR Workflow & Portfolio Deployment

What are you going to learn in this session?

In this session, you will learn how real-world collaboration happens on GitHub using Pull Requests (PRs), basic code review practices, and contribution workflows like forking. You will also deploy your project using GitHub Pages so your portfolio is accessible online.

By the end of the session, you will be able to propose changes through PRs, review code, resolve conflicts, contribute to external repositories, and share a live link of your project—exactly how professional workflows operate.

Why is this topic important?

Writing code is only one part of development. In teams, changes are proposed, reviewed, discussed, and then merged. Pull Requests formalize this process and improve code quality through collaboration.

Deployment is equally important because your work must be accessible to users and recruiters. A live portfolio with a clean repository and clear history significantly improves your chances during interviews.

How should you prepare yourself to attend this session?

Before attending, ensure your project is pushed to GitHub and you are comfortable with branches, commits, and basic push/pull operations. Keep your repository open in the browser and your terminal ready.

Be prepared to follow a full workflow: create a branch, push it, open a PR, review changes, and merge. This session is hands-on and requires you to execute each step.

What should you do after the session?

After the session, use PRs for all your changes instead of committing directly to the main branch. Practice reviewing your own PRs before merging to build strong habits.

You should also keep improving your portfolio repository by refining structure, adding a proper README, and ensuring your deployed site works correctly on different devices.

Class Notes

Pull Requests (Core Concept)

A Pull Request is a way to propose changes from one branch to another, usually from a feature branch to the main branch. It allows others to review your changes before they become part of the main codebase.

A typical flow looks like this: you create a branch, make changes, commit them, push the branch to GitHub, and then open a PR from that branch to main.

```
Shell
git switch -c feature-update
# make changes
git add .
git commit -m "update feature"
git push origin feature-update
```

Once pushed, you create a PR on GitHub and review the changes through the interface. This becomes the central place for discussion and approval.

Code Review Basics

Code reviews help improve quality, catch mistakes early, and maintain consistency across the codebase. Even simple feedback can prevent bugs and improve readability.

When reviewing a PR, focus on a few key aspects that add real value:

- Readability of the code and structure of changes
- Correctness of logic and expected behavior
- Naming clarity for variables, classes, and files

On GitHub, you can add comments directly on specific lines, suggest changes, and approve or request modifications before merging.

Merge Conflicts on GitHub

Conflicts can also occur when merging through GitHub if the same lines were changed in different branches. GitHub shows these conflicts and allows you to resolve them either through the UI or locally.

You will see conflict markers similar to local Git. After resolving, you commit the fix and complete the merge.

```
None
<<<<<< HEAD
current change
=====
incoming change
>>>>>> branch-name
```

Conflicts are a normal part of teamwork. The key is to understand what changed and choose the correct version.

Issues (Task Tracking Basics)

GitHub Issues are used to track tasks, bugs, or ideas related to a project. They help teams organize work and assign responsibilities.

You can create an issue, assign it to someone, and add labels to categorize it. This creates a simple workflow for managing development tasks without external tools.

Using Issues regularly helps you think in terms of tasks and deliverables rather than random changes.

Forking & Contribution Workflow

Forking allows you to create your own copy of someone else's repository in your GitHub account. This is commonly used in open-source contributions where you do not have direct write access.

The typical contribution flow involves forking the repository, cloning your fork locally, creating a branch, making changes, and then opening a PR to the original repository.

```
Shell
git clone <your-fork-url>
git switch -c fix-issue
```

```
# make changes
git add .
git commit -m "fix issue"
git push origin fix-issue
```

You then open a PR from your fork to the original repository. This is how most open-source contributions are made.

GitHub Pages Deployment

GitHub Pages allows you to host static websites directly from your repository. This is perfect for deploying your portfolio without needing a separate hosting service.

To deploy, go to your repository settings, enable GitHub Pages, and select the branch and folder (usually main and root). After a short time, your site will be live.

Your live URL will look like this:

```
None
https://your-username.github.io/repository-name/
```

Always verify that your site loads correctly and all links and assets work as expected.

Portfolio Repository Best Practices

Your GitHub repository is often reviewed by recruiters, so it should reflect clean and professional work. A well-maintained repository signals discipline and attention to detail.

Make sure your repository includes:

- A clean and logical folder structure
- Meaningful commit messages showing your progress
- A README file explaining your project

A simple README example:

```
None
# My Portfolio
```

```
This is my personal portfolio website built using HTML and CSS.
```

```
## Features
```

- Responsive layout
- Clean UI

```
## Live Demo
```

```
https://your-username.github.io/repository-name/
```

Interview Prep Recap

At this stage, you should be able to confidently talk about your project and the workflow you followed. Interviewers often care more about your understanding than perfect code.

You should be comfortable explaining:

- How you structured your HTML and styled it with CSS
- How you used Git for version control and branching
- How you collaborated using GitHub workflows

Practicing these explanations will help you present your work clearly during interviews.

Common Beginner Confusions

Many learners get confused between fork and clone. Fork creates a copy on GitHub, while clone downloads a repository to your local system.

PR workflows can feel complex initially, but they become intuitive with practice. Deployment issues are also common, especially when file paths or settings are incorrect.

Key Takeaways

You now understand how collaboration happens using Pull Requests, how to review code, and how to contribute using forks. You have also learned how to deploy your project using GitHub Pages and make it accessible online.

You are now at a stage where you can build, manage, collaborate, and showcase your work like a developer. The next step is consistent practice and refining your portfolio to reflect your best work.